

Introducción a Python

En estos días a principios del 2025, Python es el lenguaje de programación más popular, según el índice de TIOBE, quienes además le han dado el premio de “lenguaje de programación del año 2024”. El premio se otorga al lenguaje con el mayor incremento de calificación en un año, creciendo un 9.3% solo en el año 2024 [<https://www.tiobe.com/tiobe-index/>]. Actualmente, Python no tiene rival en varios campos de la computación, por ejemplo, análisis de datos, aprendizaje automático y desarrollo de aplicaciones de aprendizaje profundo. Una razón de este éxito se deba a lo práctico que resulta aprender el lenguaje, ya que es interpretado, lo que nos permite probar el código sin necesidad de compilarlo. Algunas desventajas son su bajo rendimiento en comparación con lenguajes compilados como C++ o Rust y que muchos de los errores en el código suceden al momento de ejecución.

En mi caso, llegué al lenguaje atraído por sus *frameworks* para el desarrollo de aplicaciones web a principios de los dos mil, por ejemplo el framework CherryPy con el que desarrollé algunas aplicaciones web, de una manera muy fácil para la época. Ya después surgieron Django y FastAPI más utilizados en la actualidad. Aunque probé otros lenguajes de scripts populares para esas tareas como tcl, PHP y ya después Ruby, la sintaxis de Python se me hizo más amigable y fácil de entender. Con el paso del tiempo surgieron muchas librerías y servicios innovadores para el desarrollo de aplicaciones científicas que no encontraba en los otros lenguajes, por lo que lo fui utilizando para otro tipo de proyectos. Por ejemplo, en 2008, Python era el único lenguaje utilizado por Google App Engine, permitiendo de una manera muy sencilla de ejecutar código de alto desempeño en la nube de manera automática. Otro proyecto interesante fue PiCloud, este servicio, permitía de una manera transparente ejecutar código de alto desempeño en la nube a partir de código ejecutado localmente. Solamente importando una librería. Era la primera aproximación a lo que hoy es computación Serverless.

Igualmente, otros programadores llegarán a interesarse por Python por algún proyecto o problema que quieren atacar, y para el cual ya existe una librería o código escrito que soluciona gran parte del problema central. Tal vez sea utilizar un API de inteligencia artificial generativa, procesar imágenes para subirlas a un servicio en la nube, programar un rastreador web, o un simulador utilizando ecuaciones diferenciales ordinarias. Python tiene librerías y ejemplos para este tipo de problemas.

Si este es tu caso, este libro está pensado en tí. El objetivo es facilitar a los programadores ponerse al día de manera rápida con el lenguaje. Servir como guía para resolver distintos tipos de problemas de ingeniería para los que Python será de gran ayuda.

Instalación de Python

En caso de que no tengas instalado y configurado Python en tu sistema aquí están unas guías para tu sistema operativo. Para hacer estas instalaciones utilizaremos la línea de comandos porque es la manera más práctica de mostrarlo en texto y es una de las maneras más prácticas de instalación para los programadores. El sistema operativo que recomiendo para programar en Python es Linux o macOS, precisamente porque están más orientados a la línea de comandos por lo que hay muchas herramientas para este estilo de trabajo. También se puede utilizar Windows pero hay algunos programas o librerías que corren nativamente en Linux o macOS y no tienen soporte oficial en Windows. Por ejemplo, Redis.

Python en tu sistema

Dependiendo de tu sistema operativo o los programas que tengas previamente instalados, es probable que haya varias instalaciones de Python en tu computadora. Por ejemplo, algunas versiones de Linux y macOS utilizan Python como parte de las herramientas del sistema operativo, por lo que tienen alguna versión preinstalada. Para complicar las cosas, de seguro va a incluir una versión más vieja, como la 2.x la cual es una versión completamente distinta y no compatible con las versiones modernas del lenguaje. Para tratar de evitar confusiones, en las versiones modernas se utiliza el nombre de `python3` para el interprete. La versión dos (vieja) se llama simplemente `python`.

Además, para los programadores, es recomendable el uso de ambientes virtuales de desarrollo, en caso de que no hayas escuchado el término, son herramientas que nos permiten tener instalaciones aisladas del lenguaje, estos ambientes incluyen sus propias copias de versiones compatibles de las librerías y código utilizados por el proyecto. Por último, también es posible programar utilizando contenedores, los cuales son en sí una instalación aislada no solo del lenguaje, si no también de los programas incluidos en el sistema operativo. Este tipo de instalaciones avanzadas las veremos más adelante. En esta sección instalaremos una versión de Python para uso general por parte de los usuarios del sistema.

Windows

En caso de que no tengas instalada la aplicación *Terminal* te recomiendo que la instales cuanto antes. Puedes instalarla desde la Microsoft Store. Esta es una versión más moderna y rápida que el tradicional programa de Command Prompt (`console.exe`), además esta versión es de código abierto.

Estos pasos para instalar Python funcionan para las versiones 10 y 11 de Windows. Para seguir practicando el uso de la terminal, vamos a utilizar la herramienta WinGet, esta herramienta nos permite gestionar aplicaciones desde la línea de comandos, parecido a lo que hacen los programas `apt` en distribuciones de Linux Debian o el programa `brew` en macOS.

Veamos si está instalado WinGet ejecutando el siguiente comando en Terminal:

```
winget -v
```

Se debería desplegar la versión del programa winget en tu computadora. Si el comando no es reconocido debemos instalar WinGet. Una vez instalado *winget*, podemos consultar las versiones de Python disponibles:

```
winget search --id Python.Python
```

Lo que nos arroja las versiones disponibles:

Nombre	Id	Versión	Origen
Python 2	Python.Python.2	2.7.18150	winget
Python 3.0	Python.Python.3.0	3.0.1	winget
Python 3.1	Python.Python.3.1	3.1.4	winget
Python 3.10	Python.Python.3.10	3.10.11	winget
Python 3.11	Python.Python.3.11	3.11.9	winget
Python 3.12	Python.Python.3.12	3.12.9	winget
Python 3.13	Python.Python.3.13	3.13.2	winget
Python 3.2	Python.Python.3.2	3.2.5	winget
Python 3.3	Python.Python.3.3	3.3.5	winget
Python 3.4	Python.Python.3.4	3.4.4	winget
Python 3.5	Python.Python.3.5	3.5.4	winget
Python 3.6	Python.Python.3.6	3.6.8	winget
Python 3.7	Python.Python.3.7	3.7.9	winget
Python 3.8	Python.Python.3.8	3.8.10	winget
Python 3.9	Python.Python.3.9	3.9.13	winget

Vamos a instalar la versión 3.13.2, la más reciente al día de hoy. Para esto utilizaremos el comando `install`, pasaremos el identificador del paquete (`--id Python.Python.3.13`) de manera exacta (`-e`). Queremos instalar Python para todos los usuarios de nuestro equipo por lo que agregaremos el parámetro `--scope machine`. Para ejecutar el comando es importante ejecutar el programa de *Terminal* como usuario **Administrador**.

```
winget install -e --id Python.Python.3.13 --scope machine
```

Para revisar que el intérprete se instaló correctamente debemos de cerrar la terminal y abrirla de nuevo para que se cargue la configuración. Una vez hecho esto ejecutamos el intérprete con el comando `python`.

```
PS C:\Users\Usuario> python
```

```
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb 4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on v
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Linux

Python se incluye en la mayoría de las distribuciones de Linux, puedes consultar distrowatch.com para ver la versión exacta incluida en tu versión específica de Linux. Por ejemplo, actualmente uso la versión Ubuntu LTS 24.04 la cual incluye la versión 3.12.3. Utilizar la versión que se incluye en los repositorios oficiales tiene la ventaja de que son versiones relativamente recientes y la mayoría de los programas y librerías son compatibles. Dicho esto, en ocasiones queremos probar la última funcionalidad del lenguaje, en ese caso podemos instalar la última versión del lenguaje desde repositorios no oficiales. Veamos las dos opciones:

Instalación utilizando APT

Antes de instalar podemos revisar si el interprete está instalado y es una versión reciente:

```
python3 --version
```

Si se imprime una versión mayor a 3.12.1 es una versión reciente y para la mayoría de los ejercicios del libro es más que suficiente.

En caso de que requieras actualizar a la última versión del interprete disponible para tu versión de Ubuntu podemos seguir los siguientes pasos:

Como primer paso, actualizamos el índice local del repositorio de paquetes de ubuntu para asegurarnos de instalar la última versión disponible.

```
sudo apt update
```

Ahora podemos actualizar nuestro sistema:

```
sudo apt -y upgrade
```

El parámetro `-y` nos permite confirmar con anticipación la instalación de las nuevas versiones.

Si solo queremos actualizar el interprete y sus dependencias podemos ejecutar el comando:

```
sudo apt install python3
```

Una vez actualizado el sistema podemos ver la versión del interprete con el siguiente comando, recuerda que debemos especificar que es la versión 3:

```
python3 --version
```

Esto debe imprimir la versión del interprete.

Ahora vamos a actualizar el programa `pip` con el que gestionamos los *paquetes* o librerías de terceros ya sea a nuestra instalación del sistema o el ambiente de desarrollo que estemos utilizando. Esto lo hacemos utilizando `pip`:

```
python3 -m pip install --upgrade pip
```

Instalación utilizando el PPA deadsnakes

Para instalar la última versión de Python en Ubuntu, vamos a utilizar el repositorio del equipo *deadsnakes* en el cual se incluyen versiones más recientes que en la distribución oficial. Para esto agregamos el repositorio a nuestro sistema con los siguientes comandos:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
```

Una vez agregado el repositorio no oficial, podemos instalar la versión más reciente. Vamos a instalar otras herramientas para desarrollo utilizando dev para incluir los encabezados de C para compilar librerías y venv para instalar las herramientas de ambientes virtuales de Python que utilizaremos más adelante:

```
sudo apt install python3.13 python3.13-venv
```

Para ejecutar la nueva versión debemos especificar la versión para ejecutar el interprete correcto:

```
python3.13
```

Para instalar `pip` para el nuevo interprete debemos ejecutar el módulo `ensurepip`:

```
python3.13 -m ensurepip --upgrade
```

Y por último actualizamos `pip`:

```
python3.13 -m pip install --upgrade pip
```

MacOS

Para instalar Python en MacOS, puedes bajar la última versión de [python.org] e instalarla utilizando la aplicación.

Introducción al lenguaje

Empezemos directamente con un ejemplo de código analizando las diferencias que vemos respecto a otros lenguajes compilados como *C#* o *Java*:

```
x = 34 - 23          # Comentario
y = "Hola"           # Otro
z = 3.45

if z == 3.45 or y == "Hola":
    x = x + 1
    y = y + " Mundo" # Concatenación de cadenas

print(x)
print(y)
```

Echando un vistazo general al programa, nos llama primero la atención el hecho de que no se importan algunas librerías al inicio del programa. Por ejemplo, para la entrada y salida de datos a la consola. También notamos que al declarar las variables no especificamos explícitamente cual es su tipo de dato. Tampoco tenemos que insertar nuestro código en algún método especial como `main` o agregarlo dentro de alguna clase. Funciona como un `script`. Veamos línea por línea.

Los nombres se atan a objetos

En la primera línea vemos la declaración de una variable llamada `x` a la cual se le asigna el resultado de una resta entre dos números enteros. Aquí encontramos la primera diferencia: en Python las “variables” no tienen un tipo de dato, son simplemente nombres o etiquetas que hacen referencia a objetos. Los objetos por su parte, sí tienen un tipo de dato. Entonces, en esta línea, el resultado de la operación se almacena en memoria en una dirección específica, una referencia. Lo que sucede entonces es que el nombre `x` se ata a la referencia del objeto entero que se crea de la operación `34-23` el `11` en la memoria. Esto es distinto al concepto de variable en otros lenguajes. Por ejemplo, en `C#` las variables `tipo valor` reservan un espacio en la memoria donde se guarda literalmente el valor que contienen, y los valores deben ser del tipo correspondiente. Si yo declaro una variable como entera de 16 bits, solo puedo almacenar en ella objetos de este tipo.

Volviendo al ejemplo, el tipo de dato al que hace referencia el nombre `x` después de la asignación, es temporalmente `int`, pero en otro momento, podríamos atar el nombre `x` a un objeto diferente de otro tipo de dato. Por ejemplo, `x = 2.3` o `x = "Hola"`. De nuevo, los objetos en memoria son tipo `float` y `string` respectivamente y nuestra etiqueta `x` se puede atar a cualquiera de ellos sin ningún problema. Vemos entonces que se atan los nombres `y` y `z` a la cadena de caracteres `"Hola"` y al flotante `3.44` respectivamente.

Cuidado Python es dinámico

Algunos programadores pueden ver esta funcionalidad de Python como algo peligroso y realmente lo es. Podemos equivocarnos fácilmente pensando que `x` hace referencia a un objeto de tipo entero cuando por algún error puede referirse a una cadena de caracteres o a un flotante y causar problemas en nuestro código. El lenguaje nos protege hasta cierto punto ya que lanzará una excepción en caso de que no esté definida una operación para un tipo de dato específico, pero en general es algo que debemos tener en cuenta en este tipo de lenguajes dinámicos. Otro problema que tenemos es que a diferencia de los lenguajes fuertemente tipados, nuestras herramientas de programación en ocasiones no pueden ayudarnos desplegando, por ejemplo, la lista de atributos o miembros de un objeto, ya que no sabe a que tipo de objeto hará referencia el nombre que estamos utilizando.

Otra diferencia importante la encontramos en este bloque de código:

```
if z == 3.45 or y == "Hola":  
    x = x + 1  
    y = y + " Mundo"  # Concatenación de cadenas  
print(x)  
print(y)
```

El espacio es importante

Aquí definimos un bloque que se va a ejecutar si la condición del `if` es verdadera. A diferencia de otros lenguajes, aquí definimos el bloque utilizando espacios (una sangría). Es común llamarle **indentación** a esta sangría por el nombre que recibe en inglés. Entonces, el bloque inicia al cambiar la indentación y debe mantenerse al saltar de línea y termina cuando regresamos la indentación al nivel anterior.

Definición de bloques con indentación consistente

En el ejemplo vemos que la condición no tiene indentación, pero el bloque de código que se ejecutará en caso de ser verdadera consta de dos líneas, estas líneas tienen una indentación consistente de cuatro espacios cada una. Para terminar el bloque, simplemente escribimos una nueva línea que no contenga los espacios. Por ejemplo, la instrucción `print(x)`.

Cuatro espacios es buen estilo

La indentación puede hacerse utilizando la tecla tabulador (tab), pero se recomienda que no se te utilicen caracteres tabulación y en su lugar se reemplace automáticamente por cuatro espacio. Esta es una capacidad que tienen los editores de código y por lo regular se hace por defecto para los archivos con extensión `.py` utilizada para los scripts de python. La convención de utilizar cuatro PEP 8, en dónde podemos encontrar las convenciones utilizadas por los programadores de las librerías estándar de distribución principal de Python. Las siglas PEP vienen del inglés *Python Enhancement Proposal* (Propuesta de Mejora para Python). Un PEP es un documento público que brinda información a la comunidad sobre alguna nueva característica o sugerencia de mejora para el lenguaje.

Siguiendo con el código, vemos también que los operadores lógicos (`and`, `or`, `not`) son palabras y no símbolos como en ciertos lenguajes. Al igual que en la mayoría de los lenguajes la concatenación de cadenas de caracteres se hace con el operador de adición. Para operaciones con otro tipo de objetos los operadores (`+` `-` `*` `/`) funcionan como siempre.

Baterías incluidas

Otra característica importante es que la función para imprimir en la consola es `print` y no tuvimos que agregar una librería para acceder a su funcionalidad. Ya viene de fábrica. De hecho, Python incluye muchas funciones de este tipo dentro del lenguaje. Utilizaremos muchas de ellas más adelante. Esto es parte del lema de **baterías incluidas** de Python, que se refiere a que el lenguaje incluye una extensa y muy útil colección de librerías (módulos en Python) en la distribución estándar.

Nota: en este texto utilizo un léxico que debería de entender un programador con algo de experiencia. Recuerda que este es el público al que va dirigido el libro. En caso de que haya algunos pocos términos que no tengas claro su significado, no hay problema, investiga un poco y te debería quedar claro.

Tipos de datos básicos

Enteros A diferencia de otros lenguajes que incluyen distintos tipos de datos para representar a enteros de distintos tamaños, por ejemplo en C# el tipo de dato `long` representa a un entero de 64 bits con signo, mientras que `short` representa a un entero con signo de 16 bits. En Python solo tenemos un tipo de dato entero `int` con una precisión arbitraria, capaz de almacenar enteros de cualquier tamaño. Bueno, el tamaño solo está limitado por la cantidad de memoria disponible en el sistema. Python evita el desbordamiento asignando dinámicamente más memoria para almacenar el resultado de una operación entera. Este costo computacional adicional hace que Python sea apropiado para el computo científico sacrificando las ventajas en desempeño que supone el tener enteros de longitud fija.

Números con punto flotante Los números con punto flotante, comúnmente llamados *flotantes* o *floats* incluyen un punto decimal y también son de tamaño arbitrario. Se pueden representar utilizando la notación exponencial (*E*) indicando a la décima potencia. Por ejemplo, $21.3E-4$ es equivalente a $21.3 * 10^{-4}$.

Cadenas o Strings En Python una cadena o string es una *secuencia* de caracteres. Una *secuencia* es una abstracción que veremos más adelante, cuando nos enfoquemos en estructuras de datos ahí veremos otros aspectos importantes de este tipo de dato. Por lo pronto podemos definir a una cadena como una colección ordenada de caracteres útil para almacenar texto. Las cadenas se pueden representar de distintas maneras. Utilizando comillas inglesas ("Hola"), comillas simples ('Hola') y esto permite evitar cierto tipo de conflictos, por ejemplo, la cadena "Carl's Jr." utiliza comillas inglesas para evitar el conflicto con la comilla simple que es parte del nombre *Carl's*. Cuando tenemos múltiples párrafos o necesitamos utilizar los dos tipos de comillas en el texto, utilizamos comillas triples, ya sean inglesas o simples:

```

"""
Este es un ejemplo del uso de
comillas triples para definir texto
que puede incluir comillas como `Carl's Jr.` y
saltos de línea.
    Indentación
    Indentación
"""

```

En otra sección nos vamos a concentrar en la funcionalidad de los objetos tipo *string* python incluye muchos métodos para realizar operaciones sobre este tipo de datos.

Booleanos Los valores de verdad en Python son representados explícitamente con los valores literales **True** o **False**, verdadero y falso respectivamente, ambos deben iniciar con mayúscula. El tipo de dato `bool` es un subtipo de `int`, por lo que en contextos numéricos **True** equivale a uno y **False** a cero respectivamente. También existen otras formas implícitas de representación de estos valores de verdad cuando se utilizan operaciones lógicas. Por ejemplo, una cadena vacía es equivalente a **False** y una colección de datos con cierto número de elementos, es equivalente a **True**. Veremos ejemplos de esto más adelante.

Comentarios

Python no cuenta con una sintaxis para definir comentarios que abarquen múltiples líneas. En este caso se pueden utilizar comillas triples para comentar múltiples líneas. Si un *valor literal* no se asigna a un nombre, este es ignorado por el intérprete de Python.

Este tipo de comentarios también se utiliza para documentar el código en situaciones específicas. Por ejemplo, se puede incluir una cadena de documentación en la primera línea de una función o clase.

```

def suma(x, y):
    """El docstring. Esta función
    regresa la suma o concatenacion de dos cadenas, es
    es importante ya que blah blah blah."""
    return x + y # Comentario aqui...

```

Los comentarios de Python son línea por línea y se especifican con el símbolo de almohadilla (`#`). En una línea, todo lo que esta a la derecha de `#` es un comentario.

Literales

En los ejemplos anteriores utilizamos *valores literales* de los tipos de datos que ilustramos. Es importante reconocer a estos valores representan datos u obje-

tos específicos de cierto tipo de dato. Entonces en los ejemplos anteriores, le definíamos nombres, mediante la asignación (o atado) a ellos de un valor literal.

Para terminar esta sección vamos a representar valores literales de los tipos de datos básicos de Python:

- Entero (`int`) : 791926378172346918273469128374619283 , 2
- Flotante (`float`): 34.3, 0.33
- Cadena (`str`): "Python es un lenguaje dinámico", 'Juan', "Leí el libro 'Pedro Páramo'."
- Booleano (`bool`): True, False

El valor especial None

La constante `None` es utilizada para indicar la ausencia de valor o un valor nulo. Tiene su propio tipo `NoneType`. De manera similar al valor `NULL` en bases de datos, el valor `None` no es igual a cero, `False` o vacío, aunque se considera `False` en condiciones booleanas. Se puede utilizar para definir un nombre que no hace referencia a un objeto todavía. Por ejemplo:

```
x = None
print(x)
```

En este caso `x` existe aunque el objeto al que está atado es `None`. Esto significa que no está atada a un valor todavía. El interprete no imprime nada cuando imprimimos `None`.

Funciones

Veamos una definición básica de una función que suma dos números:

```
>>> def suma(a, b):
...     """ Esta función suma dos números """
...     return a + b
...
>>>
```

Este ejemplo lo estoy capturando directamente en el interprete básico de Python, ejecutado de manera interactiva. Una vez en el prompt `>>>`, puedes empezar a capturar la función. Debes escribir el encabezado (la primera línea) y después dar un salto de línea con la tecla *Enter*. Si te fijas aparecen ahora tres puntos (...) en lugar del prompt, debes ahora ingresar cuatro espacios manualmente para agregar la indentación. Al final del bloque vamos a terminar de capturar la función dando un salto de línea final sin agregar espacios. Esto le dice al interprete que ya terminamos de definir el bloque y por lo tanto la función. Fíjate que el prompt regresa al modo normal `>>>`. Esto significa también que no hubo error en la captura de la función y ya se encuentra en la memoria lista para ser utilizada.

En otros lenguajes debemos de ser muy específicos indicando el tipo de dato que regresa una función o indicar de alguna manera, por ejemplo, con `void`, si la función no devuelve ningún valor.

Por ejemplo, en C:

```
1 int suma(int a, int b) {  
2     return a + b;  
3 }
```

En Python no es necesario indicar el tipo de dato que regresa la función ni tampoco el tipo de dato de los parámetros que recibe. Entonces el encabezado de la función se simplifica mucho, ya que solo indicamos con `def` el inicio de la definición de la función. Indicamos el nombre (`suma`) y entre parentesis una lista opcional de parámetros separados por una coma (`suma(a, b)`). Por último, algo muy importante, el símbolo de dos puntos (`:`) que marca el inicio del bloque de código que define el cuerpo de la función.

Como vimos anteriormente, el bloque del cuerpo de la función debe estar indentado utilizando cuatro espacios. El bloque termina con la única instrucción de `return a + b` indicando que queremos regresar como resultado la operación `a + b`. La suma de los dos números `a` y `b`.

Una vez definida la función, podemos ejecutarla de esta manera, utilizando valores literales como parámetros:

```
>>> suma(3, 4)  
7
```

En este ejemplo no estamos asignando el resultado de la suma a ninguna variable o imprimiendo el resultado. Sin embargo, el ejemplo esta pensado para ejecutarse de manera interactiva utilizando el intérprete de este modo. En este caso, el resultado de la función se imprimiría automáticamente en la siguiente línea de la sesión. Por otro lado, si escribimos el código como un script y lo corremos no se imprimiría nada, pues no se ejecuta de manera interactiva.

Para ver el resultado si ejecutamos el script podríamos escribir algo como:

```
# Esto está en un archivo llamado programa.py  
# Lo puedes ejecutar con el intérprete así: python3 programa.py  
resultado = suma(3, 4)  
print(resultado)  
  
# Aún más compacto  
print(suma(1,2))
```

Hay un detalle importante en nuestro código. Por alguna razón estamos asegurando que `a` y `b` son valores numéricos. ¿Pero que hay del caso que se muestra a continuación?

```
>>> suma('hola', ' mundo')
'hola mundo'
```

El resultado sería 'hola mundo', la concatenación de las dos cadenas de entrada. Esto resalta el punto que habíamos considerado anteriormente: No debemos asumir que los nombres estarán atados a objetos de cierto tipo.

Desde la versión 3.5 se le agregó la anotación de tipos (*type hints* o *type annotations*). Estas anotaciones nos permiten especificar como en otros lenguajes, el tipo de dato de los parámetros y el del valor que regresan las funciones. El problema es que no son obligatorias y no tienen un efecto en tiempo de ejecución. Python no vigila que los datos enviados o regresados sean los correctos. Ejemplo:

```
def suma(a: int, b: int) -> int:
    return a + b
```

Aquí, `a` y `b` se anotan como enteros (`int`), y `-> int` indica que la función devuelve un entero. Estas anotaciones son utilizadas por herramientas de edición y librerías para alertarnos de manera estática de posibles errores en nuestro código. En otra sección veremos el uso del método `type()` incluido de fábrica para inspeccionar en tiempo de ejecución el tipo de dato de una variable, lo que nos permitirá verificar que los parámetros sean del tipo correcto. Por lo pronto puedes investigar este método e intentar validar que solo se reciban y regresen valores enteros.

En el caso de que no especifiquemos una instrucción de `return` la función regresará el valor de `None` de manera automática.

Ejemplo de una versión interactiva de esta sección:

```
Python 3.11.5 (main, Sep 11 2023, 08:31:25) [Clang 14.0.6 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> def suma(a, b):
...     return a + b
...
>>> suma(2, 3)
5
>>> suma('hola ', 'mundo')
'hola mundo'
>>> exit()
```

Las funciones son ciudadanos de primera clase

Cuando decimos que en un lenguaje de programación las funciones son ciudadanos de primera clase (*first-class citizens*), significa que las funciones tienen el mismo estatus que otros tipos de datos como los enteros, cadenas o flotantes.

Esto significa que las funciones:

- Se pueden asignar a variables
- Se pueden pasar como parámetros a otras funciones
- Se pueden regresar como resultado de otras funciones
- Se pueden almacenar en estructuras de datos como listas, diccionarios, etc.

Este es un concepto fundamental en la programación funcional que veremos más adelante. Veamos un ejemplo:

```
>>> def por_tres(a):
      return a * 3

>>> def aplica(f, a):
      return f(a)

>>> aplica(por_tres, 7)
21
```

Este código se está ejecutando de manera interactiva por eso se incluye el prompt `>>>` del intérprete. En la primera instrucción definimos la función `por_tres`, esta función triplica el valor que entra como parámetro. Ahora viene lo bueno. La función `aplica` toma como parámetro una función (`f`) y regresa el resultado de pasar como parámetro a esa función el parámetro `a`. Aquí vemos el concepto de *primera clase*, podemos tomar como parámetro de una función otra función. Esto lo hacemos en la tercera instrucción `aplica(por_tres, 7)`. Enviamos a la función `aplica` la función `por_tres` y nos regresa como resultado el resultado de la función, en este caso `7 * 3`. Este tema lo vamos a retomar cuando veamos el tema de programación funcional.